CS3000 Practice Quiz 5C — Answer Key

New DP problems: perfect squares and minimum hours

Notation follows the class notes on dynamic programming: $C(v)$ (coin change), $K(i, w)$ (0/1 knapsack), $L(i, j)$ (LCS), $D(i, j)$ (edit distance); strings and items are 1-indexed. Every table below was computed by a program and checked against brute force. Shaded cells mark the traceback path. Point splits imitate the Gradescope rubrics used on Quizzes 1–4 (each question 5 pts, 10 total).

Q1. Fewest perfect squares

(a) The last square used is some $j^2 \le n$; the rest is an optimal sum for $n - j^2$:

$$S(n) = \begin{cases} 0 & n = 0 \\ \min\limits_{1 \le j,\ j^2 \le n} \left(1 + S(n - j^2)\right) & n > 0 \end{cases}$$

This is exactly coin change with denominations $1, 4, 9, \ldots$ (all squares $\le n$).

(b) Table.

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13
$S(n)$	0	1	2	3	1	2	3	4	2	1	2	3	3	2

e.g. $S(12) = \min(1 + S(11), 1 + S(8), 1 + S(3)) = \min(4, 3, 4) = 3$; $S(13) = \min(1 + S(12), 1 + S(9), 1 + S(4)) = \min(4, 2, 2) = 2$.

(c) The smallest failure is $n = \mathbf{12}$: greedy takes $9+1+1+1$ (**4** squares) but $S(12) = \mathbf{3}$ $(4+4+4)$.

(d)

```
function MIN-SQUARES (n):
    S = new array [0..n]
    S[0] = 0
    for a from 1 to n+1:          // index n+1 non-inclusive
        S[a] = infinity
        j = 1
        while j * j <= a:
            S[a] = min(S[a], 1 + S[a - j*j])
            j = j + 1
    return S[n]
```

The inner loop runs $\lfloor \sqrt{a} \rfloor \le \sqrt{n}$ times, so the time is $\Theta(n\sqrt{n})$; space $\Theta(n)$.

Rubric (5 pts).

- **+1 pt** Recurrence over squares $j^2 \le n$ with $S(0) = 0$.
- **+1.5 pts** Table correct (-0.5 per wrong entry, min 0).
- **+1 pt** $n = 12$: greedy 4 vs. optimal 3.
- **+1.5 pts** Bottom-up pseudocode and $\Theta(n\sqrt{n})$ running time.

Q2. Minimum hours for at least $P = 8$ points

(a)

$$H(i,k) = \begin{cases} 0 & k = 0 \\ \infty & i = 0,\ k > 0 \\ \min\big(H(i-1,k),\ h_i + H(i-1, \max(0, k-p_i))\big) & i \geq 1,\ k > 0 \end{cases}$$

Skip A_i, or do it: it covers p_i of the k points, and if $p_i \geq k$ nothing more is needed (hence $\max(0, \cdot)$).

(b) Table (shaded = traceback path).

$i\downarrow\ k\rightarrow$	0	1	2	3	4	5	6	7	8
$i = 0$	**0**	∞	∞	∞	∞	∞	∞	∞	∞
$i = 1$	**0**	5	5	5	5	5	5	∞	∞
$i = 2$	0	2	2	**2**	5	5	5	7	7
$i = 3$	0	2	2	2	4	4	5	6	**6**
$i = 4$	0	2	2	2	3	4	5	5	**6**

(c) $H(4,8) =$ **6** hours. $H(4,8) = H(3,8)$, so A_4 is skipped. $H(3,8) = 6 \neq H(2,8) = 7$, so A_3 is used; $k = 8 - 5 = 3$. $H(2,3) = 2 \neq H(1,3) = 5$, so A_2 is used; $k = 0$. Do **A_2 and A_3**: $2 + 4 = 6$ hours for $3 + 5 = 8$ points (the unique optimum).

(d) Same take-it-or-leave-it structure over items (0/1 choice, two-dimensional table, $\Theta(nP)$ time and space). Different: it *minimizes* cost subject to *at least* P points (a covering constraint) instead of maximizing value subject to *at most* W weight, so the base case for unreachable states is ∞ (not 0) and overshooting is clamped to 0.

Rubric (5 pts).

+1.5 pts Recurrence with ∞ base case and the $\max(0, k - p_i)$ clamp.

+1.5 pts Table correct (-0.5 per wrong row).

+1 pt 6 hours via A_2, A_3, with traceback.

+1 pt Clear comparison to 0/1 knapsack.